Web Technologies Exam Preparation Guide

PUCIT / FCIT — SEF24 — ASP.NET Core MVC, C#, ADO.NET & Bootstrap 5

Personalized revision handbook for BS Software Engineering

October 2026

Table of Contents

Evidence Map and How to Use This Guide

What this document is

This is a **course revision handbook** for your Web Technologies exam at the University of the Punjab (PUCIT/FCIT), taught by **Professor Shuja Ur Rehman Baig**. It follows your actual labs and homework, uses a consistent **Car Rental** example, and prepares you for both conceptual and hand-written coding questions.

Honest research note (read this)

ChatGPT conversation history was not accessible in this Cursor session. This guide was built from:

Source	Status	What it confirms
SEF24-Web-Lab-01.pdf	Confirmed	C# console, byte/character streams, FileStream, StreamWriter/StreamReader
SEF24-Web-Lab-02-Updated.docx	Confirmed	Classes/encapsulation, **ADO.NET**, SQL Server, ProductDAL, CRUD, parameterized SQL
SEF24-Web-Lab-03.docx	Confirmed	Safe search (anti SQL injection), JSON ser/deser, **disconnected model** (DataTable, SqlDataAdapter)

Source	Status	What it confirms
web_hw1.pdf (HW 4)	Confirmed	ASP.NET Core MVC Car Rental, Bootstrap 5.3, layouts, GET/POST, cards/tables; **no EF Core, DB, repos, or interfaces for that HW**
lecture6/ (Student, StudentDAL)	Partial	Same DAL naming pattern as Lab 02 (files were empty stubs)
Prior ChatGPT Web chats	**Not available**	Topics you listed in the master prompt (ViewModels, DI, async, layouts/sections, etc.) are included as **exam prep aligned with your stated learning goals**, clearly labelled

Do not claim EF Core was your taught data-access stack. Labs 02–03 use **ADO.NET + SQL Server**. HW 4 uses **in-memory lists**. Repository/interfaces/DI appear in later conceptual chapters as patterns that build on your DAL work — verify with your teacher if they are on the written exam.

Topic classification

1. Confirmed topics studied (course materials)

- C# classes, properties, constructors, encapsulation, `ToString()`
- Byte streams vs character streams; file I/O
- ADO.NET connected model: `SqlConnection`, `SqlCommand`, `SqlDataReader`
- Parameterized queries and SQL injection awareness
- Full CRUD through a **DAL** class
- Disconnected model: `DataTable`, `SqlDataAdapter`, `SqlCommandBuilder`
- JSON Serialize / Deserialize
- ASP.NET Core MVC: models, controllers, strongly typed views
- GET/POST actions, Post/Redirect/Get style booking flow
- Shared `_Layout.cshtml`, Bootstrap 5.3 navbar, cards, tables, alerts, badges
- In-memory `List<Car>` as temporary data store (HW 4)

2. Related / mentioned but not deeply evidenced in files

- ViewModels, `ViewBag`/`ViewData`/`TempData` (common MVC; not in HW text)
- Tag Helpers and data annotations (typical MVC form work)
- Dependency Injection and interfaces (HW 4 explicitly forbade interfaces)
- `async`/`await` with database calls (natural extension of ADO.NET)

- Repository pattern (your DAL is the same idea under a different name)

3. Syllabus gaps to verify with your teacher

- Entity Framework Core (HW 4 says **do not use**)
- Unit of Work
- ASP.NET Core Web APIs
- Authentication / authorization
- Deployment and hosting
- Whether async programming is examinable
- Whether DI lifetimes (`Transient`/`Scoped`/`Singleton`) are examinable

Exam strategy (teacher-oriented)

1. **Define** in one sentence (exam answer box in each chapter).
2. **Compare** related ideas in a table (GET/POST, List/IEnumerable, connected/disconnected).
3. **Write syntax by hand** — practice without IntelliSense (Rider will not help in the exam hall).
4. **Trace** a request: Browser → Routing → Controller → DAL/Repository → DB → View → HTML.
5. **Know HW 4 rules** separately from "full architecture" patterns: HW 4 = in-memory, no interfaces; later chapters = how production MVC usually grows.

Consistent example

Throughout this guide we use **Car Rental**:

- Domain model: `Car` with `Id`, `Name`, `PricePerDay`
- Controllers: `CarsController`
- Views: Available cars (cards), Booked cars (table)
- Layout: dark responsive Bootstrap navbar

When we add database chapters, the same `Car` maps to a SQL Server table via ADO.NET — the technology your labs actually taught.

Chapter 1: C# and .NET Foundations Relevant to Web Development

1.1 Class, object, instance, and interface

Simple definition: A **class** is a blueprint. An **object** (also called an **instance**) is a live copy of that blueprint in memory. An **interface** is a contract that says *what* methods a type must provide, without saying *how*.

Why needed: Web apps need models (`Car`, `Product`), data helpers (`ProductDAL`), and later services. Clear types keep exam code readable.

Intuition (Roman Urdu tip): Class = ghar ka map. Object = actual ghar. Interface = rule book: "door aur window hona zaroori hai," bina yeh bataye ke wood se banao ya steel se.

Syntax:

```
public class Car
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public decimal PricePerDay { get; set; }
}

public interface ICarRepository
{
    List<Car> GetAvailable();
}
```

Exam answer: A class defines structure and behaviour; an object is a concrete instance of a class; an interface defines a contract that implementing classes must fulfil.

1.2 Properties, fields, methods, constructors, access modifiers

Lab evidence: Lab 02 requires private fields with public get/set properties, parameterized + parameterless constructors, and `ToString()`.

```
// Models/Car.cs  (mirrors Lab 02 Product style)
public class Car
```

```
{
    private int _id;
    private string _name = string.Empty;
    private decimal _pricePerDay;

    public int Id
    {
        get { return _id; }
        set { _id = value; }
    }

    public string Name
    {
        get { return _name; }
        set { _name = value; }
    }

    public decimal PricePerDay
    {
        get { return _pricePerDay; }
        set { _pricePerDay = value; }
    }

    // Parameterless — needed for object initializers and model binding
    public Car() { }

    // Parameterized
    public Car(int id, string name, decimal pricePerDay)
    {
        Id = id;
        Name = name;
        PricePerDay = pricePerDay;
    }

    public override string ToString()
    {
        return $"[{Id}] {Name} — PKR {PricePerDay}/day";
    }
}
```

Keyword	Meaning
public	Visible everywhere

Keyword	Meaning
private	Only inside this class
protected	This class + derived classes
get / set	Read / write property accessors
override	Replace a base-class method (ToString)

Object initializer (Lab 02):

```
var c = new Car { Id = 1, Name = "Alto", PricePerDay = 3000m };
```

Common mistakes: Forgetting parameterless constructor when using { Name = ... }; making fields public instead of properties (breaks encapsulation requirement in lab).

Practice: Write a Product class matching Lab 02 columns from memory.

1.3 Namespaces and using

Definition: A **namespace** groups related types. A **using** directive imports a namespace so you can write short type names.

```
using System.Data.SqlClient;   // or Microsoft.Data.SqlClient

namespace CarRental.Models
{
   public class Car { /* ... */ }
}
```

In ASP.NET Core MVC, folders often mirror namespaces: Controllers, Models, ViewModels.

1.4 Generics, List<T>, and IEnumerable<T>

Definition: Generics let you write type-safe collections like List<Car> instead of untyped lists.

Type	Meaning	Typical use
List<T>	Concrete, resizable list;	DAL return type, in-memory HW 4

Type	Meaning	Typical use
	can Add/Remove/index	lists
IEnumerable<T>	“Something you can foreach”; read-oriented contract	Method parameters, flexible return type

```
List<Car> available = new List<Car>();
available.Add(new Car(1, "Alto", 3000m));

IEnumerable<Car> viewData = available; // List implements IEnumerable
foreach (Car car in viewData)
{
   Console.WriteLine(car.Name);
}
```

Why program to IEnumerable sometimes? Caller only needs to loop; you can later change the concrete type. **Why List in DAL?** Lab 02 explicitly returns List<Product> — fine for exams.

Exam answer: List<T> is a concrete collection with Add/Remove; IEnumerable<T> is an interface for iterating a sequence. Every List<T> is an IEnumerable<T>, but not every IEnumerable<T> is a List<T>.

1.5 Interface vs concrete class; programming to interfaces

```
public interface IProductDal
{
   List<Product> GetAllProducts();
}

public class ProductDAL : IProductDal
{
   public List<Product> GetAllProducts() { /* ADO.NET ... */ return list; }
}
```

	Class	Interface
Has meth	Yes	No (until default interface methods — avoid in exam)

	Class	Interface
Holds data?		
Can new it?	Yes	No
Purpose	Implementation	Contract

HW 4 note: Teacher said *do not use interfaces* for that lab. Still learn the idea — viva and later exams often ask.

1.6 Inheritance and polymorphism (light)

```
public class Vehicle { public string Name { get; set; } = ""; }
public class Car : Vehicle { public decimal PricePerDay { get; set; } }
```

Polymorphism: treat a `Car` as a `Vehicle` when only shared members are needed. Your labs focus more on **composition** (DAL uses Product) than deep inheritance.

1.7 Exceptions and basic error handling

```
try
{
   // open connection, execute command
}
```

```
catch (SqlException ex)
{
    Console.WriteLine("Database error: " + ex.Message);
}
finally
{
    // dispose / close if not using `using`
}
```

Prefer using for connections (auto-dispose):

```
using (SqlConnection conn = new SqlConnection(connectionString))
{
    conn.Open();
}
```

1.8 Value types vs reference types (exam-relevant)

Value types	Reference types
int, decimal, bool, struct	class, string, arrays, List<T>
Copied by value	Variable holds a reference

string is a reference type but often feels immutable. Models like Car are reference types — two variables can point to the same car object.

1.9 .NET, CLR, managed code, assemblies (overview)

Term	Short definition
.NET	Platform + libraries for building apps
CLR	Common Language Runtime — runs managed code, GC, JIT
Managed code	Code executed by the CLR
Assembly	Compiled unit (.dll or .exe)
DLL	Library assembly reused by apps
Executable	Entry-point assembly (OutputType Exe in console labs)

Your Lab 02 project is a console **Exe** targeting a .NET SDK (net9.0 appears in local lecture stubs). ASP.NET Core MVC apps also compile to assemblies hosted by a web server (Kestrel).

Chapter 1 practice set

Q1. Define class vs object vs interface in three lines.
Q2. Why does Lab 02 ask for both constructors?
Q3. Difference between `List<Car>` and `IEnumerable<Car>`?
Q4. Write a `Product` class with private fields and public properties.
Q5. Find the error: `IProductDal dal = new IProductDal();`

Answer key (Ch 1)

1. Class = blueprint; object/instance = living copy; interface = contract of members without implementation.

2. Parameterized for explicit creation; parameterless for object initializers / binding.

3. List is concrete and mutable with indexes; IEnumerable only guarantees enumeration.

4. Mirror Lab 02: ProductId, Name, Price, QuantityInStock, Category.

5. Cannot instantiate an interface; use `new ProductDAL()`.

Chapter 2: Web Fundamentals and HTTP

2.1 Client–server architecture

Simple definition: The **client** (usually a browser) requests resources; the **server** processes the request and sends a response.

Roles:

Piece	Responsibility
Browser	UI, sends HTTP, renders HTML/CSS/JS
Web server / Kestrel	Receives HTTP, runs ASP.NET Core
MVC application	Routing, controllers, views,

Piece	Responsibility
	business rules
Database (SQL Server)	Persistent storage (Labs 02–03; not HW 4)

```
Browser  --HTTP request-->  ASP.NET Core MVC  --SQL-->  SQL Server
Browser  <--HTML/redirect--  ASP.NET Core MVC  <--rows-- SQL Server
```

2.2 HTTP request and response

Request parts: method (GET/POST), URL/path, query string, headers, optional body.
Response parts: status code, headers, body (HTML, JSON, empty for redirect).

Example GET:

```
GET /Cars/Index HTTP/1.1
Host: localhost:5000
```

Example POST (book a car):

```
POST /Cars/Book HTTP/1.1
Content-Type: application/x-www-form-urlencoded

id=2
```

2.3 GET vs POST

	GET	POST
Purpose	Read / display	Change state (book, save, delete)
Data location	Query string / URL	Request body
Idempotent?	Should be safe to repeat	Not for bookings
Caching	Often cacheable	Usually not
HW 4	Index, Booked	Book

Exam answer: GET retrieves data without changing server state; POST submits data that may change state (for example booking a car).

2.4 URL, route, query string, body, headers, status codes

- **URL:** https://localhost:5001/Cars/Booked
- **Route:** /Cars/Booked → controller Cars, action Booked
- **Query string:** /Cars/Details?id=3 → id=3
- **Body:** form fields on POST
- **Headers:** Content-Type, cookies, etc.

Code	Meaning	Typical MVC use
200	OK	return View(...) success
302	Found / Redirect	RedirectToAction(...)
400	Bad Request	BadRequest()
404	Not Found	NotFound()
500	Server Error	Unhandled exception

2.5 How a browser request reaches ASP.NET Core MVC

1. User clicks link or submits form
2. Browser sends HTTP request to Kestrel
3. Middleware pipeline runs (routing, etc.)
4. Routing matches /Cars/Available → CarsController.Available
5. Action runs (may call DAL/repository)
6. Action returns IActionResult (View / Redirect / ...)
7. If View: Razor renders HTML (often inside _Layout)
8. Response HTML returned to browser
9. Browser paints page; Bootstrap CSS/JS style and behave

HW 4 booking flow (Post/Redirect/Get):

GET /Cars/Index → show cards
POST /Cars/Book → move car between lists
302 Redirect to Booked
GET /Cars/Booked → show table

Refreshing after redirect does **not** re-POST the book action — that is why PRG matters.

2.6 Basic form / database security (course-aligned)

1. **SQL injection:** never concatenate user input into SQL — Lab 02/03 require parameters.

2. **Antiforgery:** MVC forms include a hidden token against CSRF (Chapter 6).

3. **Validate input** before writing to DB.

4. **Least privilege** connection string (concept; supply your own credentials — never invent secrets in notes).

Chapter 2 practice

Q1. Draw the request path for `/Cars/Booked`.
Q2. Why is booking a POST, not a GET?
Q3. What does status 302 mean after `Book`?
Q4. Difference between query string and form body?

Answer key

1. Browser → server → route to CarsController.Booked → View → HTML.

2. Booking changes state; GET should be safe/read-only.

3. Redirect response; browser then GETs Booked.

4. Query string is in the URL; body is in the POST payload.

Chapter 3: ASP.NET Core MVC Architecture

3.1 What MVC means

Simple definition: MVC separates an app into **Model** (data), **View** (HTML UI), and **Controller** (handles requests and chooses responses).

Why: Controllers stay thin; views stay for display; models/DAL own data. Easier exam tracing and maintenance.

Intuition: Restaurant — Customer (browser) talks to Waiter (controller). Waiter gets food info from Kitchen/stock (model/DAL). Plate presentation is the View.

Part	Responsibility	HW 4 example
Model	Data shape	Car
View	Razor .cshtml	Views/Cars/ Index.cshtml
Controller	Actions	CarsController

3.2 Typical project folders

```
CarRental/
  Program.cs          # app startup, services, middleware
  Controllers/
    CarsController.cs
  Models/
    Car.cs
  Views/
    _ViewStart.cshtml
    _ViewImports.cshtml
    Shared/
      _Layout.cshtml
    Cars/
      Index.cshtml
      Booked.cshtml
  wwwroot/            # static CSS, JS, images
  appsettings.json      # config (connection strings later)
```

3.3 Controllers, actions, and naming

```
// Controllers/CarsController.cs
using Microsoft.AspNetCore.Mvc;
using CarRental.Models;

public class CarsController : Controller
{
    // HW 4 style — in-memory (resets on restart)
    private static List<Car> availableCars = new()
    {
        new Car { Id = 1, Name = "Alto", PricePerDay = 3000m },
```

```
        new Car { Id = 2, Name = "Corolla", PricePerDay = 6000m },
        new Car { Id = 3, Name = "Civic", PricePerDay = 8000m }
    };
    private static List<Car> bookedCars = new();

    [HttpGet]
    public IActionResult Index()
    {
        return View(availableCars); // Views/Cars/Index.cshtml
    }

    [HttpPost]
    public IActionResult Book(int id)
    {
        Car? car = availableCars.FirstOrDefault(c => c.Id == id);
        if (car == null)
            return RedirectToAction(nameof(Index));

        availableCars.Remove(car);
        bookedCars.Add(car);
        return RedirectToAction(nameof(Booked));
    }

    [HttpGet]
    public IActionResult Booked()
    {
        return View(bookedCars);
    }
}
```

Conventions:

- Class name ends with `Controller`
- Action method name maps to view name by default
- Route `/Cars/Index` → `CarsController.Index`

3.4 How `/Cars/Available` would be processed

If you had an action `Available`:

1. Route values: `controller=Cars`, `action=Available`
2. Framework creates `CarsController` (via DI)

3. Calls Available()
4. return View() looks for Views/Cars/Available.cshtml
5. Renders with layout from _ViewStart

3.5 Return types: IActionResult vs ViewResult

Type	Meaning
IActionResult	Interface for any action result (view, redirect, content, 404…)
ViewResult	Specific result that renders a Razor view

Exam tip: Prefer IActionResult as the declared return type so one action can return View() **or** RedirectToAction().

```
public IActionResult Details(int id)
{
   if (id <= 0) return BadRequest();
   var car = availableCars.FirstOrDefault(c => c.Id == id);
   if (car == null) return NotFound();
   return View(car);          // ViewResult under the hood
}
```

Method	Effect
return View()	View named like the action
return View("Details")	Named view Details.cshtml
return View(model)	Strongly typed model to default view
RedirectToAction("Booked")	302 to another action
Redirect("/home")	302 to raw URL
Content("ok")	Plain text
NotFound()	404
BadRequest()	400

When to return a view vs redirect:
- View: show a page (GET).
- Redirect: after successful POST (PRG), or when car not found → back to Index.

3.6 Complete request lifecycle diagram

```
Browser
  |  GET /Cars/Index
  v
Kestrel + Middleware
  v
Routing  -->  CarsController.Index()
  v
(optional) DAL / Repository / DB
  v
return View(availableCars)
  v
Razor: Views/Cars/Index.cshtml + Shared/_Layout.cshtml
  v
HTML + Bootstrap CSS/JS
  v
Browser paints "Available Cars"
```

3.7 Program.cs (conceptual)

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();
// Later: builder.Services.AddScoped<ICarRepository, CarRepository>();

var app = builder.Build();
if (!app.Environment.IsDevelopment())
{
   app.UseExceptionHandler("/Home/Error");
}
app.UseStaticFiles();
app.UseRouting();
app.MapControllerRoute(
   name: "default",
   pattern: "{controller=Cars}/{action=Index}/{id?}");
app.Run();
```

Chapter 3 exam practice

Short: What does MVC stand for?
Difference: IActionResult vs ViewResult.
Syntax: Write Book POST that redirects to Booked.
Trace: User opens /Cars/Booked — which files run?
Error: return View("Index", bookedCars) from Booked — which view?

Answer key

- Model-View-Controller.
- Interface vs specific view result.
- See code above.
- Routing → Booked action → Views/Cars/Booked.cshtml + layout.
- Still uses Index.cshtml because of the name argument; model is booked list.

Chapter 4: Razor Syntax and Data Passing

4.1 What is Razor?

Definition: Razor is the markup language for MVC views: HTML mixed with C# using @. Files use extension .cshtml.

Why: Build dynamic pages (lists of cars) without separate templating engines.

4.2 @model vs @Model

Syntax	Meaning
@model List<Car>	**Directive** at top: declares the view's type (compile-time)
@Model	**Property**: the actual object instance passed from the controller

```
@* Views/Cars/Index.cshtml *@
@model List<CarRental.Models.Car>
```

```
<h2>Available Cars</h2>
<p>Select a car to book.</p>

@if (Model == null || !Model.Any())
{
    <div class="alert alert-info">No cars available.</div>
}
else
{
    <div class="row">
    @foreach (var car in Model)
    {
        <div class="col-md-4 mb-3">
            <div class="card h-100 shadow-sm">
                <div class="card-body">
                    <h5 class="card-title">@car.Name</h5>
                    <p class="card-text">PKR @car.PricePerDay / day</p>
                    <span class="badge bg-success">Available</span>
                </div>
            </div>
        </div>
    }
    </div>
}
```

Matching controller:

```
public IActionResult Index()
{
    return View(availableCars); // type must match @model
}
```

4.3 Expressions and code blocks

```
@* Expression — writes encoded value *@
<p>@car.Name</p>

@* Code block — no automatic output *@
@{
    var title = "Available Cars";
    var count = Model.Count;
```

```
}
<h1>@title (@count)</h1>
```

4.4 Conditionals and loops

```
@if (Model.Count == 0)
{
    <div class="alert alert-info">Empty</div>
}
else if (Model.Count == 1)
{
    <p>Only one car left.</p>
}
else
{
    <p>Many cars.</p>
}

@for (int i = 0; i < Model.Count; i++)
{
    <li>@Model[i].Name</li>
}
```

4.5 ViewBag, ViewData, TempData

Bag	Type	Lifetime	Survives redirect?
ViewBag	dynamic	Current request	No
ViewData	dictionary	Current request	No
TempData	dictionary	Until read (often next request)	Yes (one hop)

```
public IActionResult Index()
{
    ViewBag.PageTitle = "Available Cars";
    ViewData["Count"] = availableCars.Count;
    return View(availableCars);
}

public IActionResult Book(int id)
```

```
{
    TempData["Message"] = "Car booked!";
    return RedirectToAction(nameof(Booked));
}
```

```
<h1>@ViewBag.PageTitle</h1>
<p>Count: @ViewData["Count"]</p>
@if (TempData["Message"] != null)
{
    <div class="alert alert-success">@TempData["Message"]</div>
}
```

Prefer strongly typed models / ViewModels for exam clarity; bags are shortcuts.

4.6 Passing a single model vs a list

```
return View(car);              // @model Car
return View(availableCars);    // @model List<Car>  or IEnumerable<Car>
```

Booked table view (HW 4):

```
@model List<CarRental.Models.Car>

<div class="d-flex justify-content-between align-items-center mb-3">
    <h2>Booked Cars</h2>
    <a class="btn btn-secondary" asp-action="Index">Back to Cars</a>
</div>

@if (Model.Count == 0)
{
    <div class="alert alert-info">No bookings yet.</div>
}
else
{
    <div class="table-responsive">
        <table class="table table-striped table-hover">
            <thead class="table-dark">
                <tr>
                    <th>ID</th><th>Car Name</th><th>Price Per Day</th><th>Status</th>
                </tr>
            </thead>
            <tbody>
            @foreach (var car in Model)
```

```
        {
            <tr>
                <td>@car.Id</td>
                <td>@car.Name</td>
                <td>@car.PricePerDay</td>
                <td><span class="badge text-bg-warning">Booked</span></td>
            </tr>
        }
        </tbody>
    </table>
</div>
}
```

4.7 Common Razor mistakes

1. Forgetting `@model` → `Model` is untyped/dynamic issues.
2. Writing `@model` twice with different types — **illegal**.
3. Using `Model.Name` when `@model` is `List<Car>` — lists have no `Name`.
4. Missing `@` before C# identifiers.
5. Putting heavy business logic in the view (prefer controller/DAL).

Exam answer: `@model` declares the view's type; `@Model` accesses the instance passed by `return View(data)`.

Chapter 5: ViewModels and Multiple Objects

> **Evidence note:** HW 4 only passes `List<Car>`. ViewModels are standard MVC exam material and match topics you listed as studied with ChatGPT; confirm with your teacher if they appear on the paper.

5.1 The problem

A view may need:

- list of available cars

- total car count
- page heading
- a success message flag

Razor allows **only one** @model directive. You cannot write:

```
@model List<Car>
@model string   @* ERROR — not allowed *@
```

5.2 Solution: ViewModel

Definition: A **ViewModel** is a class designed for one screen. It combines several properties the view needs into **one** object.

Domain model vs ViewModel:

Domain model (Car)	ViewModel (AvailableCarsViewModel)
Mirrors business/DB entity	Mirrors **page** needs
Reused across app	Often one view (or a few)
Id, Name, PricePerDay	Cars, TotalCount, Heading

5.3 Create a ViewModel

```
// ViewModels/AvailableCarsViewModel.cs
using CarRental.Models;

namespace CarRental.ViewModels
{
    public class AvailableCarsViewModel
    {
        public string Heading { get; set; } = "Available Cars";
        public int TotalCount { get; set; }
        public List<Car> Cars { get; set; } = new();
    }
}
```

5.4 Controller passes ViewModel

```
[HttpGet]
public IActionResult Index()
{
    var vm = new AvailableCarsViewModel
    {
        Heading = "Available Cars",
        Cars = availableCars,
        TotalCount = availableCars.Count
    };
    return View(vm);
}
```

5.5 View uses ViewModel

```
@model CarRental.ViewModels.AvailableCarsViewModel

<h2>@Model.Heading</h2>
<p>Total: @Model.TotalCount</p>

@if (Model.Cars.Count == 0)
{
    <div class="alert alert-info">No cars available.</div>
}
else
{
    @foreach (var car in Model.Cars)
    {
        <div>@car.Name — PKR @car.PricePerDay</div>
    }
}
```

5.6 ViewModel vs ViewBag / ViewData

Approach	Type-safe?	Refactor-friendly?	Exam clarity
ViewModel	Yes	Excellent	Best
ViewBag	Weak	Poor	OK for demos
ViewData	Cast needed	Poor	Avoid for multi-data

Intuition: ViewModel = ek dabba jisme page ki sari cheezen packed hain.

5.7 Forms and ViewModels

When a form needs extra fields (customer name + car id), make a form ViewModel:

```
public class BookCarViewModel
{
   public int CarId { get; set; }
   [Required]
   public string CustomerName { get; set; } = "";
}
```

(HW 4 does not require customer details — keep that distinction clear.)

Exam answer

A ViewModel is a presentation-focused class that packages all data one view needs into a single object so the view can stay strongly typed with one `@model` directive.

Practice: Design a ViewModel for Booked page with `Cars`, `Heading`, and `HasBookings` bool.

Chapter 6: HTML Forms, Tag Helpers, GET, POST, and Model Binding

High priority for HW 4 and exams.

6.1 HTML form basics

Definition: A **form** collects user input and sends it to the server with GET or POST.

Element	Role
form	Container; action = URL; method = get/post

Element	Role
label	Accessible caption for an input
input	text, number, hidden, submit
select	dropdown
textarea	multi-line text
submit button	triggers send

```
<form action="/Cars/Book" method="post">
  <input type="hidden" name="id" value="2" />
  <button type="submit">Book Car</button>
</form>
```

Field names **model binding:** name="id" binds to action parameter int id or property Id.

6.2 GET forms vs POST forms

	GET form	POST form
Data in	Query string	Body
Use	Search/filter	Create/update/book
HW 4	—	Book button

6.3 [HttpGet] and [HttpPost]

```
[HttpGet]
public IActionResult Create() => View();

[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Create(Car car)
{
   if (!ModelState.IsValid)
      return View(car); // redisplay form with errors

   // save...
   return RedirectToAction(nameof(Index));
}
```

Same action name, different HTTP verb — common pattern.

6.4 Model binding

Definition: ASP.NET Core copies request values into parameters or model properties by **name matching**.

```
// Primitive
public IActionResult Book(int id) { }

// Complex model
public IActionResult Create(Car car) { }
```

Form:

```
<input name="Name" value="Alto" />
<input name="PricePerDay" value="3000" />
```

binds to car.Name and car.PricePerDay.

6.5 Tag Helpers (MVC)

Tag Helper	Purpose
asp-controller	Sets controller for link/form
asp-action	Sets action
asp-for	Binds label/input to model property + generates name/id
asp-validation-for	Shows validation message for a property
asp-route-id	Route value

@model CarRental.Models.Car

```
<form asp-controller="Cars" asp-action="Create" method="post">
  <div class="mb-3">
    <label asp-for="Name" class="form-label"></label>
    <input asp-for="Name" class="form-control" />
    <span asp-validation-for="Name" class="text-danger"></span>
  </div>
  <div class="mb-3">
    <label asp-for="PricePerDay" class="form-label"></label>
    <input asp-for="PricePerDay" class="form-control" />
```

```
    <span asp-validation-for="PricePerDay" class="text-danger"></span>
  </div>
  <button type="submit" class="btn btn-primary">Save</button>
</form>
```

6.6 HW 4 book form (complete, with filenames)

Controller — Controllers/CarsController.cs (excerpt):

```
[HttpPost]
public IActionResult Book(int id)
{
  var car = availableCars.FirstOrDefault(c => c.Id == id);
  if (car == null)
    return RedirectToAction(nameof(Index));

  availableCars.Remove(car);
  bookedCars.Add(car);
  return RedirectToAction(nameof(Booked));
}
```

View — Views/Cars/Index.cshtml (card with POST):

```
<form asp-action="Book" method="post">
  <input type="hidden" name="id" value="@car.Id" />
  <button type="submit" class="btn btn-primary w-100">Book Car</button>
</form>
```

Trace when user books Corolla (Id = 2)

1. Browser shows Index (GET) with three cards.
2. User clicks Book on Corolla.
3. POST /Cars/Book with body id=2 (+ antiforgery token if Tag Helpers used).
4. Model binder sets id = 2.
5. Controller finds Corolla, removes from availableCars, adds to bookedCars.

6. Returns **302** RedirectToAction("Booked").

7. Browser GETs /Cars/Booked.

8. Table shows Corolla with yellow “Booked” badge.

If Id not found: redirect Index, lists unchanged.

6.7 Validation: ModelState and data annotations

```
using System.ComponentModel.DataAnnotations;

public class Car
{
   public int Id { get; set; }

   [Required]
   [StringLength(100)]
   public string Name { get; set; } = "";

   [Range(1, 100000)]
   public decimal PricePerDay { get; set; }
}

if (!ModelState.IsValid)
   return View(car);
```

Accepting data ≠ saving data: validate first; only then insert via DAL.

6.8 Antiforgery

Tag Helper forms emit a hidden __RequestVerificationToken. On POST, add:

[ValidateAntiForgeryToken]

Protects against Cross-Site Request Forgery.

6.9 Common mistakes

1. Using GET for Book (unsafe; can be triggered by crawler).
2. Wrong `name` on input → parameter stays 0/null.
3. Forgetting redirect after POST → refresh re-submits.
4. Returning View after successful POST instead of Redirect.
5. `asp-for` on a view without matching `@model`.

Exam answer: Model binding maps request values to action parameters/properties by name; POST actions that change state should redirect (PRG) after success.

Chapter 7: Database Connectivity and SQL (ADO.NET)

High priority. Confirmed stack from **Lab 02** and **Lab 03**: **ADO.NET** with **SQL Server**, organized in a **DAL** class. **EF Core was not part of your labs; HW 4 forbids it.**

7.1 Relational basics

Term	Meaning
Table	Collection of rows (e.g. Products, Cars)
Row	One record
Column	One field
Primary key	Unique id (ProductId IDENTITY)
Foreign key	Reference to another table (not required in Lab 02)

CRUD: Create (INSERT), Read (SELECT), Update (UPDATE), Delete (DELETE).

Lab 02 table:

```
CREATE TABLE Products (
  ProductId INT PRIMARY KEY IDENTITY(1,1),
```

```
    Name VARCHAR(100) NOT NULL,
    Price DECIMAL(10,2) NOT NULL,
    QuantityInStock INT NOT NULL,
    Category VARCHAR(50) NOT NULL
);
```

Car-rental equivalent (you supply DB name/credentials yourself):

```
CREATE TABLE Cars (
    Id INT PRIMARY KEY IDENTITY(1,1),
    Name VARCHAR(100) NOT NULL,
    PricePerDay DECIMAL(10,2) NOT NULL,
    IsBooked BIT NOT NULL DEFAULT 0
);
```

7.2 What is ADO.NET?

Definition: ADO.NET is .NET's classic API for talking to relational databases using connections, commands, and readers/adapters.

Why: Direct SQL control; matches your course labs exactly.

Important classes (Lab 02/03):

Class	Role
SqlConnection	Connection to SQL Server
SqlCommand	SQL text + parameters
SqlDataReader	Forward-only connected read
SqlParameter	Safe parameter value
SqlDataAdapter	Fill/update DataTable (disconnected)
SqlCommandBuilder	Auto-generate update commands for adapter
DataTable / DataRow	In-memory table/row

Namespaces: System.Data, System.Data.SqlClient (or Microsoft.Data.SqlClient on modern .NET — same ideas).

7.3 Connection strings

Pattern only — put your real server/login in config; do not invent passwords in exam notes as if they were real.

Server=YOUR_SERVER;Database=ProductDB;User Id=YOUR_USER;Password=YOUR_PASSWORD;TrustServerCertificate=True;

Or Windows auth style (if teacher uses it):

Server=.;Database=ProductDB;Integrated Security=True;TrustServerCertificate=True;

Store in a private field on the DAL (Lab 02).

7.4 Connected model (Lab 02)

Definition: Keep a connection open while you execute and read (SqlDataReader).

Flow: Open connection → Execute command → Read rows → Close/dispose.

```
// ProductDAL.cs — pattern from Lab 02
using System.Data.SqlClient;

public class ProductDAL
{
    private readonly string _connectionString;

    public ProductDAL(string connectionString)
    {
        _connectionString = connectionString;
    }

    private SqlConnection GetConnection()
    {
        return new SqlConnection(_connectionString);
    }

    // CREATE
    public void AddProduct(Product product)
    {
        const string sql = @"INSERT INTO Products (Name, Price, QuantityInStock, Category)
                             VALUES (@Name, @Price, @Qty, @Category)";
        using SqlConnection conn = GetConnection();
```

```
        using SqlCommand cmd = new SqlCommand(sql, conn);
        cmd.Parameters.AddWithValue("@Name", product.Name);
        cmd.Parameters.AddWithValue("@Price", product.Price);
        cmd.Parameters.AddWithValue("@Qty", product.QuantityInStock);
        cmd.Parameters.AddWithValue("@Category", product.Category);
        conn.Open();
        cmd.ExecuteNonQuery(); // INSERT/UPDATE/DELETE
    }

    // READ ALL
    public List<Product> GetAllProducts()
    {
        var list = new List<Product>();
        const string sql = "SELECT ProductId, Name, Price, QuantityInStock, Category FROM
Products";
        using SqlConnection conn = GetConnection();
        using SqlCommand cmd = new SqlCommand(sql, conn);
        conn.Open();
        using SqlDataReader reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            list.Add(new Product
            {
                ProductId = reader.GetInt32(0),
                Name = reader.GetString(1),
                Price = reader.GetDecimal(2),
                QuantityInStock = reader.GetInt32(3),
                Category = reader.GetString(4)
            });
        }
        return list;
    }

    // READ ONE
    public Product? GetProductById(int productId)
    {
        const string sql = @"SELECT ProductId, Name, Price, QuantityInStock, Category
                             FROM Products WHERE ProductId = @Id";
        using SqlConnection conn = GetConnection();
        using SqlCommand cmd = new SqlCommand(sql, conn);
        cmd.Parameters.AddWithValue("@Id", productId);
        conn.Open();
        using SqlDataReader reader = cmd.ExecuteReader();
```

```
        if (!reader.Read()) return null;
        return new Product
        {
            ProductId = reader.GetInt32(0),
            Name = reader.GetString(1),
            Price = reader.GetDecimal(2),
            QuantityInStock = reader.GetInt32(3),
            Category = reader.GetString(4)
        };
    }

    // UPDATE
    public void UpdateProduct(Product product)
    {
        const string sql = @"UPDATE Products
                             SET Name=@Name, Price=@Price, QuantityInStock=@Qty,
Category=@Category
                             WHERE ProductId=@Id";
        using SqlConnection conn = GetConnection();
        using SqlCommand cmd = new SqlCommand(sql, conn);
        cmd.Parameters.AddWithValue("@Name", product.Name);
        cmd.Parameters.AddWithValue("@Price", product.Price);
        cmd.Parameters.AddWithValue("@Qty", product.QuantityInStock);
        cmd.Parameters.AddWithValue("@Category", product.Category);
        cmd.Parameters.AddWithValue("@Id", product.ProductId);
        conn.Open();
        cmd.ExecuteNonQuery();
    }

    // DELETE
    public void DeleteProduct(int productId)
    {
        const string sql = "DELETE FROM Products WHERE ProductId=@Id";
        using SqlConnection conn = GetConnection();
        using SqlCommand cmd = new SqlCommand(sql, conn);
        cmd.Parameters.AddWithValue("@Id", productId);
        conn.Open();
        cmd.ExecuteNonQuery();
    }
}
```

Execute methods

Method	Returns	Use
ExecuteNonQuery	rows affected	INSERT/UPDATE/ DELETE
ExecuteReader	SqlDataReader	SELECT many/one
ExecuteScalar	first cell	COUNT(*), single value

Mapping query → objects

1. Execute reader.
2. For each row, new Product { ... }.
3. Add to List<Product>.
4. Return list to menu/controller — **SQL stays inside DAL.**

7.5 Parameterized queries and SQL injection (Lab 02 + 03)

Bad (vulnerable):

```
string sql = "SELECT * FROM Products WHERE Name LIKE '%" + keyword + "%'";
```

Attacker types: ' OR 1=1 -- and may see all rows / worse.

Good (Lab 03 SearchProductsByName):

```
public List<Product> SearchProductsByName(string keyword)
{
    var list = new List<Product>();
    const string sql = @"SELECT ProductId, Name, Price, QuantityInStock, Category
                         FROM Products WHERE Name LIKE @keyword";
    using SqlConnection conn = GetConnection();
    using SqlCommand cmd = new SqlCommand(sql, conn);
    cmd.Parameters.AddWithValue("@keyword", "%" + keyword + "%");
    // wildcards in VALUE, not concatenated into SQL text
    conn.Open();
    using SqlDataReader reader = cmd.ExecuteReader();
    while (reader.Read())
    {
```

```
        list.Add(/* map columns */);
    }
    return list;
}
```

Lab comment to remember: Parameterized commands keep user input **data**, not **executable SQL**.

7.6 Disconnected model (Lab 03)

Definition: Load data into a DataTable, close the connection, edit in memory, later push changes with SqlDataAdapter.Update.

```
public DataTable LoadProductsTable(out SqlDataAdapter adapter)
{
    adapter = new SqlDataAdapter("SELECT ProductId, Name, Price, QuantityInStock,
Category FROM Products",
                                 GetConnection());
    SqlCommandBuilder builder = new SqlCommandBuilder(adapter);
    DataTable table = new DataTable();
    adapter.Fill(table); // opens/closes connection for you
    return table;
}

public void UpdatePriceInTable(DataTable table, SqlDataAdapter adapter, int productId,
decimal newPrice)
{
    DataRow[] rows = table.Select("ProductId = " + productId);
    if (rows.Length == 0) return;
    rows[0]["Price"] = newPrice;
    adapter.Update(table); // writes back to SQL Server
}
```

Connected	Disconnected
SqlDataReader while connected	DataTable in memory
Good for quick read loops	Good for edit sets / UI grids
Lab 02 CRUD	Lab 03 price update module

7.7 JSON export/import (Lab 03)

```
using System.Text.Json;

public void ExportProductsToJson(string filePath)
{
    List<Product> products = GetAllProducts();
    string json = JsonSerializer.Serialize(products);
    File.WriteAllText(filePath, json);
}

public void ImportProductsFromJson(string filePath)
{
    string json = File.ReadAllText(filePath);
    List<Product>? products = JsonSerializer.Deserialize<List<Product>>(json);
    if (products == null) return;
    foreach (var p in products)
        AddProduct(p); // mind duplicates in real apps
}
```

7.8 Transactions (conceptual — verify if taught)

```
using SqlConnection conn = GetConnection();
conn.Open();
using SqlTransaction tx = conn.BeginTransaction();
try
{
    // several commands with CommandTransaction = tx
    tx.Commit();
}
catch
{
    tx.Rollback();
    throw;
}
```

Why: All succeed or all fail (e.g. book car + payment).

7.9 Async ADO.NET (extension for web — Chapter 9)

If using async:

- OpenAsync
- ExecuteReaderAsync
- ExecuteNonQueryAsync
- ExecuteScalarAsync

Only if your exam expects async — verify. Pattern matches sync names + Async suffix + await.

7.10 What was NOT confirmed as taught

Technology	Status
ADO.NET + SQL Server	**Confirmed** Labs 02–03
DAL class	**Confirmed**
EF Core	**Not in labs; HW 4 forbids**
Dapper	**Not evidenced**
PostgreSQL	**Not evidenced** (SQL Server in Lab 02)

7.11 Common mistakes

1. SQL in the console/controller instead of DAL.
2. String-concatenating user input into SQL.
3. Forgetting Open().
4. Not disposing connections (using).
5. Assuming ExecuteNonQuery returns the new identity (use SCOPE_IDENTITY() / scalar if needed).
6. Confusing connected reader with disconnected adapter.

Exam answer: ADO.NET uses SqlConnection and SqlCommand to run SQL; SqlDataReader reads connected results; parameterized queries prevent SQL injection; a DAL hides all of this from the UI layer.

Chapter 8: Repository Pattern and Application Architecture

8.1 Your DAL *is* the repository idea

Lab wording: ProductDAL — Data Access Layer.
Industry wording: Repository — same job: **all database code in one place**, UI/controllers never open SqlConnection.

HW 4 reminder: That assignment forbids repositories/interfaces and uses in-memory lists. This chapter is for architecture understanding beyond HW 4.

8.2 Why not put SQL in controllers?

Problem if SQL in controller	Benefit of DAL/Repository
Duplicated connection code	One place to fix bugs
Hard to test	Can mock interface later
Fat controllers	Thin actions: call method, return View
Mix HTML concerns with SQL	Clear separation

8.3 Model vs repository vs service vs controller

Layer	Job	Car Rental example
Model	Data shape	Car
Repository/DAL	CRUD SQL	CarRepository / CarDAL
Service (optional)	Business rules	“Cannot book if already booked”
Controller	HTTP in/out	CarsController

Layer	Job	Car Rental example
View	HTML	Index.cshtml

Beginner exams often merge service into controller + repository.

8.4 Folder structure (Car Rental + ADO.NET)

```
CarRental/
  Models/Car.cs
  Data/ICarRepository.cs      # optional interface
  Data/CarRepository.cs       # ADO.NET implementation
  Controllers/CarsController.cs
  Views/...
  Program.cs
```

8.5 Interface + implementation

```
// Data/ICarRepository.cs
public interface ICarRepository
{
    List<Car> GetAvailable();
    List<Car> GetBooked();
    bool Book(int id);
}

// Data/CarRepository.cs
public class CarRepository : ICarRepository
{
    private readonly string _cs;
    public CarRepository(string connectionString) => _cs = connectionString;

    public List<Car> GetAvailable()
    {
        // SELECT ... WHERE IsBooked = 0  with SqlDataReader
        // map to List<Car>
        throw new NotImplementedException("Write full SQL in exam practice");
    }

    public List<Car> GetBooked() { /* IsBooked = 1 */ throw new
NotImplementedException(); }
```

```
    public bool Book(int id)
    {
        // UPDATE Cars SET IsBooked = 1 WHERE Id=@id AND IsBooked = 0
        // return rowsAffected == 1
        throw new NotImplementedException();
    }
}
```

Controller calling repository:

```
public class CarsController : Controller
{
    private readonly ICarRepository _repo;
    public CarsController(ICarRepository repo) => _repo = repo;

    public IActionResult Index() => View(_repo.GetAvailable());

    [HttpPost]
    public IActionResult Book(int id)
    {
        if (!_repo.Book(id))
            return RedirectToAction(nameof(Index));
        return RedirectToAction(nameof(Booked));
    }

    public IActionResult Booked() => View(_repo.GetBooked());
}
```

Data flow: Browser → Controller → Repository → SQL Server → Repository maps rows → Controller → View → Browser.

8.6 Advantages and limitations

Advantages: separation of concerns, reuse, easier testing with fakes, cleaner controllers.
Limitations: extra types/files; overkill for tiny HW 4; bad repositories that leak SQL details still hurt.

Exam answer: A repository (or DAL) centralizes data access so controllers request data through methods instead of embedding SQL and connection code.

Chapter 9: Asynchronous Programming in C# and ASP.NET Core

Critical chapter. Async was not named in Lab 01–03 PDFs; it is included because your master prompt lists it as studied and it is standard for ASP.NET Core + ADO.NET. **Verify with your teacher whether hand-written async is on the exam.**

9.1 Synchronous vs asynchronous

Sync: Thread waits idle while DB responds.
Async: While waiting for I/O (database, file, network), the thread can serve other requests.

Intuition: Sync = cashier stands frozen waiting for card machine. Async = cashier helps next customer while machine processes.

Roman Urdu: Async ka matlab CPU ko bekaar khada na rakhna jab tak database jawab de — wait ke dauran doosri request handle ho sakti hai.

Async does **not** magically speed up heavy CPU math; it helps **I/O waits**.

9.2 async, await, Task, Task<T>

Keyword / type	Meaning
async	Marks method as asynchronous; allows await inside
await	Wait for a Task to finish **without blocking the thread** the same way .Result does
Task	Represents an async operation with **no return value**
Task<T>	Async operation that eventually returns a **T**

```
// Returns nothing useful to caller except "work finished"
public async Task SaveAsync()
{
   await cmd.ExecuteNonQueryAsync();
}

// Returns List<Car> when done
public async Task<List<Car>> GetAvailableAsync()
{
   // ... await reader ...
   return list;
}
```

void VS Task VS Task<T>

Return type	Use
async void	**Avoid** (except event handlers) — exceptions hard to catch
async Task	Async method with no result
async Task<T>	Async method that returns T

9.3 What await does (before / after)

```
public async Task<IActionResult> Index()
{
   // BEFORE await: still on request context
   var cars = await _repo.GetAvailableAsync();
   // AFTER await: continuation runs when DB finished
   return View(cars);
}
```

While awaiting, ASP.NET can use the thread for other requests.

9.4 Side-by-side: sync vs async MVC

Synchronous:

```
public IActionResult Index()
{
   var cars = _repository.GetAll();
   return View(cars);
}
```

Asynchronous:

```
public async Task<IActionResult> Index()
{
   var cars = await _repository.GetAllAsync();
   return View(cars);
}
```

Every layer that awaits must itself be async (propagate async all the way).

9.5 Full stack trace (interface → repository → ADO.NET → controller → view)

```
// 1) Interface
public interface ICarRepository
{
   Task<List<Car>> GetAvailableAsync();
   Task<bool> BookAsync(int id);
}

// 2) Repository with real async ADO.NET
public class CarRepository : ICarRepository
{
   private readonly string _cs;
   public CarRepository(string cs) => _cs = cs;

   public async Task<List<Car>> GetAvailableAsync()
   {
      var list = new List<Car>();
      const string sql = "SELECT Id, Name, PricePerDay FROM Cars WHERE IsBooked = 0";
      await using var conn = new SqlConnection(_cs);
      await using var cmd = new SqlCommand(sql, conn);
      await conn.OpenAsync();
      await using var reader = await cmd.ExecuteReaderAsync();
      while (await reader.ReadAsync())
      {
```

```
            list.Add(new Car
            {
                Id = reader.GetInt32(0),
                Name = reader.GetString(1),
                PricePerDay = reader.GetDecimal(2)
            });
        }
        return list;
    }

    public async Task<bool> BookAsync(int id)
    {
        const string sql = @"UPDATE Cars SET IsBooked = 1
                             WHERE Id = @id AND IsBooked = 0";
        await using var conn = new SqlConnection(_cs);
        await using var cmd = new SqlCommand(sql, conn);
        cmd.Parameters.AddWithValue("@id", id);
        await conn.OpenAsync();
        int rows = await cmd.ExecuteNonQueryAsync();
        return rows == 1;
    }
}

// 3) Controller
public class CarsController : Controller
{
    private readonly ICarRepository _repo;
    public CarsController(ICarRepository repo) => _repo = repo;

    public async Task<IActionResult> Index()
    {
        List<Car> cars = await _repo.GetAvailableAsync();
        return View(cars);
    }

    [HttpPost]
    public async Task<IActionResult> Book(int id)
    {
        bool ok = await _repo.BookAsync(id);
        if (!ok) return RedirectToAction(nameof(Index));
        return RedirectToAction(nameof(Booked));
    }
```

```
    public async Task<IActionResult> Booked()
    {
        // similar GetBookedAsync...
        return View(await _repo.GetAvailableAsync()); // replace with GetBookedAsync in
real code
    }
}
```

Where async / await are required

Layer	Must be async?	Why
ADO.NET call	Yes (OpenAsync, etc.)	Actual I/O
Repository method	Yes	awaits DB
Interface signature	Yes (Task<...>)	match implementation
Controller action	Yes	awaits repository
View	No	just displays list

Wrong: mark controller async but call sync GetAll() with no await — gains nothing.
Wrong: async method with no await — compiler warning; still sync work.

9.6 Exception handling

```
public async Task<List<Car>> GetAvailableAsync()
{
    try
    {
        // await DB
        return list;
    }
    catch (SqlException ex)
    {
        // log ex
        throw; // or return empty / rethrow domain exception
    }
}
```

await unwraps exceptions from the Task into normal catch.

9.7 Avoid .Result and .Wait() in web apps

```
// BAD — can deadlock / block thread pool
var cars = _repo.GetAvailableAsync().Result;
_repo.GetAvailableAsync().Wait();
```

Prefer await all the way up.

9.8 Concurrency vs parallelism vs async I/O (short)

Term	Idea
Async I/O	Don't block waiting for DB/network
Concurrency	Many tasks in progress (interleaved)
Parallelism	Multiple CPU cores compute together

Web DB calls → **async I/O**, not "run SQL on 8 cores."

9.9 Exam traps

1. Returning List<Car> from method declared Task<List<Car>> without Task — use return list inside async method (compiler wraps) or return Task.FromResult(list).

2. Interface sync but implementation async — signatures must match.

3. Forgetting await → you pass a Task object to the view by mistake.

4. async void action methods.

5. Mixing: controller async, repository sync only — incomplete story.

Exam answer: async/await with Task/Task<T> let web actions wait for database I/O without blocking threads; asynchronous methods should be propagated from data access through the repository to the controller.

Practice: Convert Lab 02 GetAllProducts to GetAllProductsAsync and show the controller signature change.

Chapter 10: Dependency Injection, IoC, Classes, and Interfaces

> **HW 4** forbids interfaces. Still learn DI — it is core ASP.NET Core and often viva/exam material.

10.1 Quick definitions

Term	One-line definition
Class	Blueprint for objects
Object / instance	Concrete value created from a class
Interface	Contract of members a class must implement
Dependency	Another object your class needs to work (CarsController needs a repository)
IoC (Inversion of Control)	Framework calls *your* code and supplies dependencies — you don't new everything
DI (Dependency Injection)	A technique to supply dependencies (often via constructor)

IoC vs DI: IoC is the broad principle; DI is the common way to achieve it in ASP.NET Core.

10.2 Interface syntax and implementation

```
public interface ICarRepository
{
    List<Car> GetAvailable();
}

public class CarRepository : ICarRepository
{
    public List<Car> GetAvailable() { /* ... */ return new List<Car>(); }
}
```

10.3 Problem with `new` inside a controller

```
public class CarsController : Controller
{
    private readonly CarRepository _repo = new CarRepository("..."); // tight coupling
}
```

Problems: hard to replace with test fake; controller knows concrete class; connection string plumbing messy.

10.4 Constructor injection

```
public class CarsController : Controller
{
    private readonly ICarRepository _repo;

    public CarsController(ICarRepository repo)
    {
        _repo = repo; // ASP.NET Core supplies this
    }

    public IActionResult Index() => View(_repo.GetAvailable());
}
```

10.5 Registering in `Program.cs`

```
builder.Services.AddScoped<ICarRepository, CarRepository>();
```

Meaning: when something asks for `ICarRepository`, create `CarRepository`.

Lifetimes

Method	Lifetime	Typical use
AddTransient	New instance every injection	Lightweight stateless helpers
AddScoped	One instance per HTTP request	**Repositories / DAL** (usual choice)
AddSingleton	One for whole app	Shared cache; careful with

Method	Lifetime	Typical use
		scoped deps

Mistake: injecting a **scoped** service into a **singleton** — captive dependency; scoped becomes effectively singleton.

10.6 Full path example

```
// Interface
public interface ICarRepository { List<Car> GetAvailable(); }

// Implementation
public class CarRepository : ICarRepository
{
   private readonly string _cs;
   public CarRepository(IConfiguration config)
   {
      _cs = config.GetConnectionString("Default")
          ?? throw new InvalidOperationException("Missing connection string");
   }
   public List<Car> GetAvailable() { /* ADO.NET */ return new(); }
}

// Program.cs
builder.Services.AddScoped<ICarRepository, CarRepository>();

// Controller — constructor injection shown above
```

If not registered → runtime exception when resolving `CarsController`.

10.7 Why depend on interfaces?

- Swap `CarRepository` for `FakeCarRepository` in tests
- Change SQL implementation without editing controller
- Clear contract for exam diagrams

Exam answer: Dependency Injection supplies a class's dependencies from the outside (usually constructor), often registered in Program.cs, so code depends on abstractions and is easier to test and replace.

Chapter 11: Razor Layouts, Shared Files, and Sections

High priority — HW 4 Task 3.

11.1 Why layouts?

Definition: A **layout** is a shared HTML shell (navbar, footer, CSS/JS) that wraps individual views.

Why underscore? Convention: _Layout.cshtml, _ViewStart.cshtml — files starting with _ are partial/shared helpers, not directly browsed as pages. This is mostly a **naming convention**, not magic by itself; _ViewStart / _ViewImports have **special framework behaviour**.

11.2 Special files

File	Role
Views/Shared/_Layout.cshtml	Master HTML template
Views/_ViewStart.cshtml	Sets default Layout for views in folder tree
Views/_ViewImports.cshtml	Shared @using and tag helpers
Partial e.g. _CarCard.cshtml	Reusable fragment

```
@* Views/_ViewStart.cshtml *@
@{
   Layout = "_Layout";
}
```

```
@* Views/_ViewImports.cshtml *@
@using CarRental.Models
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

11.3 @RenderBody() — HW 4 Car Rental layout

```
@* Views/Shared/_Layout.cshtml *@
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="utf-8" />
   <meta name="viewport" content="width=device-width, initial-scale=1" />
   <title>@ViewData["Title"] - Car Rental</title>
   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
         rel="stylesheet" />
   @await RenderSectionAsync("Styles", required: false)
</head>
<body>
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand" asp-controller="Cars" asp-action="Index">Car Rental</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
            data-bs-target="#navMain" aria-controls="navMain"
            aria-expanded="false" aria-label="Toggle navigation">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navMain">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item">
          <a class="nav-link" asp-controller="Cars" asp-action="Index">Available Cars</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" asp-controller="Cars" asp-action="Booked">Booked Cars</a>
        </li>
      </ul>
    </div>
  </div>
</nav>

<main class="container my-4">
   @RenderBody()
</main>

<footer class="container border-top py-3 text-center">
   All rights reserved
</footer>
```

```
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></
script>
@await RenderSectionAsync("Scripts", required: false)
</body>
</html>
```

How it wraps a view: Razor executes Index.cshtml content and injects it where `@RenderBody()` sits.

Navbar tip: With `navbar-dark`, do **not** force dark text classes on links (`text-dark`) — they become invisible.

11.4 Sections

Definition: A **section** is optional/required content a view defines and the layout renders in a named slot.

```
@* In layout *@
@await RenderSectionAsync("Scripts", required: false)
```

```
@* In Views/Cars/Index.cshtml *@
@section Scripts {
    <script>
        console.log("Index page script");
    </script>
}

@section Styles {
    <style>.card { border-radius: 0.5rem; }</style>
}
```

API	Role
`@RenderBody()`	Main view content
`@RenderSectionAsync("Name")`	Named extra content

Rule: Section is **defined in the view**, **rendered in the layout**. If `required: true` and view omits section → error.

11.5 Multiple layouts / no layout

Admin layout: Views/Shared/_AdminLayout.cshtml — different nav.

```
@{
    Layout = "_AdminLayout";
}
```

Standalone (no layout):

```
@{
    Layout = null;
}
```

11.6 Partials

@await Html.PartialAsync("_CarCard", car)

Partials usually start with _. They do not run a full layout cycle by themselves.

11.7 Bootstrap CSS/JS placement

- **CSS** in <head> of layout (once)
- **JS bundle** before </body> (once)
- Page-specific scripts in @section Scripts

HW 4: load Bootstrap **once in the layout**, not in every view.

11.8 Common errors

1. Forgetting _ViewStart → views without chrome.
2. Typo in data-bs-target vs collapse id → toggle broken.
3. Defining @section Scripts but layout has no RenderSectionAsync.

4. Putting Bootstrap JS only on one page — collapse fails elsewhere.

5. Conflicting text colour on dark navbar.

Exam answer: _Layout.cshtml provides shared page chrome; @RenderBody() inserts the view; _ViewStart sets the default layout; sections inject page-specific CSS/JS into slots in the layout.

Chapter 12: Bootstrap 5 Complete Exam Revision

Based on **HW 4** (Bootstrap **5.3**) plus common components for exams. Bootstrap styles and JS behaviour; it does **not** do server pagination, SQL, or booking logic.

12.1 Layout and grid

Class	Meaning
container	Centred fixed-width responsive wrapper
container-fluid	Full width
row	Horizontal row of columns
col	Equal flexible column
col-md-6	Half width from md breakpoint up
col-sm-12	Full width from sm up

Breakpoints (BS5): sm 576px, md 768px, lg 992px, xl 1200px, xxl 1400px.

```
<div class="container my-4">
  <div class="row">
    <div class="col-md-4">Card 1</div>
    <div class="col-md-4">Card 2</div>
    <div class="col-md-4">Card 3</div>
  </div>
</div>
```

Mistake: columns without a row parent; nesting rows incorrectly.

12.2 Spacing

Class	Meaning
m-3	margin all sides
mt-2 / mb-3	margin top / bottom
mx-auto	horizontal centre (block)
p-3	padding all
py-2 / px-4	padding y / x

Margin = outside gap; **padding** = inside gap.

12.3 Flexbox

```
<div class="d-flex justify-content-between align-items-center gap-2 flex-wrap">
  <h2>Booked Cars</h2>
  <a class="btn btn-secondary">Back to Cars</a>
</div>
```

Class	Effect
d-flex	flex container
flex-row / flex-column	direction
justify-content-center / between	main axis
align-items-center	cross axis
gap-2	gap between items
flex-wrap	allow wrap

12.4 Typography and colours

text-center, text-start, text-end, fw-bold, fst-italic, text-uppercase, h1–h6, display-1, lead, text-primary, text-danger, text-muted, bg-light, bg-dark, bg-success.

12.5 Display and sizing

d-none, d-block, d-md-block, w-100, h-50, border, border-primary, rounded, rounded-circle.

12.6 Buttons

```
<button class="btn btn-primary w-100">Book Car</button>
<a class="btn btn-secondary" href="/Cars/Index">Back</a>
```

Variants: btn-success, btn-danger, btn-warning, btn-outline-primary, btn-sm, btn-lg.

12.7 Cards (HW 4 available cars)

```
<div class="card h-100 shadow-sm">
  <div class="card-body d-flex flex-column">
    <h5 class="card-title">Corolla</h5>
    <p class="card-text">PKR 6000 / day</p>
    <span class="badge bg-success mb-2">Available</span>
    <form method="post" action="/Cars/Book" class="mt-auto">
      <input type="hidden" name="id" value="2" />
      <button class="btn btn-primary w-100" type="submit">Book Car</button>
    </form>
  </div>
</div>
```

h-100 + mt-auto keeps buttons aligned in a grid.

12.8 Tables (HW 4 booked)

```
<table class="table table-striped table-hover">
  <thead class="table-dark">...</thead>
  <tbody>...</tbody>
</table>
```

Wrap in table-responsive for small screens.

12.9 Forms

```
<div class="mb-3">
  <label class="form-label">Name</label>
  <input class="form-control" />
</div>
<select class="form-select">...</select>
```

12.10 Alerts and badges

```
<div class="alert alert-info">No cars available.</div>
<span class="badge bg-success">Available</span>
<span class="badge text-bg-warning">Booked</span>
```

HW 4: yellow booked badge with dark text → text-bg-warning (or bg-warning text-dark).

12.11 Navbar (HW 4)

See Chapter 11 full example. Key pieces:

- navbar navbar-expand-lg navbar-dark bg-dark
- Brand left: Car Rental

- navbar-toggler + data-bs-toggle="collapse" + data-bs-target="#navMain"
- Collapse div **same id** navMain
- Needs Bootstrap **JS bundle** for toggle

12.12 List groups

```
<ul class="list-group list-group-flush">
  <li class="list-group-item">Alto</li>
  <li class="list-group-item">Civic</li>
</ul>
```

Also: list-group-numbered, list-group-horizontal.

12.13 Pagination (UI only)

```
<ul class="pagination">
  <li class="page-item disabled"><a class="page-link">Prev</a></li>
  <li class="page-item active"><a class="page-link">1</a></li>
  <li class="page-item"><a class="page-link">2</a></li>
  <li class="page-item"><a class="page-link">Next</a></li>
</ul>
```

Server must still skip/take rows — Bootstrap only draws controls.

12.14 Modals

```
<button class="btn btn-primary" data-bs-toggle="modal" data-bs-
target="#confirmModal">
  Open
</button>
<div class="modal fade" id="confirmModal" tabindex="-1">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title">Confirm</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal"></button>
      </div>
      <div class="modal-body">Book this car?</div>
      <div class="modal-footer">
        <button class="btn btn-secondary" data-bs-dismiss="modal">Close</button>
        <button class="btn btn-primary">Yes</button>
      </div>
    </div>
  </div>
</div>
```

Needs JS bundle for data-bs-toggle / dismiss.

12.15 Dropdowns

```
<div class="dropdown">
  <button class="btn btn-secondary dropdown-toggle" data-bs-
toggle="dropdown">Menu</button>
  <ul class="dropdown-menu">
    <li><a class="dropdown-item" href="#">Item</a></li>
  </ul>
</div>
```

12.16 What belongs in layout vs view

In _Layout	In individual views
Bootstrap CSS + JS	Page heading, cards, tables
Navbar, footer	Forms for that page
@RenderBody()	Optional @section Scripts

Exam answer: Bootstrap 5 is a CSS/JS toolkit for responsive layout and components; MVC still owns routing, data, and booking logic.

Chapter 13: Other Topics from Course History

13.1 Lab 01 — Streams and file handling (confirmed)

Byte stream: FileStream + Encoding.UTF8.GetBytes / decode buffer → books_byte.dat.
Character stream: StreamWriter / StreamReader.ReadLine() → books_char.txt.

Difference (exam): Byte streams work with raw bytes; character streams work with text and encodings more conveniently. Both can store the same book records; APIs differ.

Menu: Add Book, View via Byte, View via Character, Exit.

13.2 .NET / CLR

See Chapter 1. Console labs compile to managed assemblies; MVC apps run under ASP.NET Core on the CLR/Kestrel.

13.3 ADO.NET summary pointer

Chapter 7 is the full treatment. Remember: **DAL only**, parameterized SQL, connected vs disconnected.

13.4 SQL Server (confirmed) vs others

Lab 02 names **SQL Server** (SqlConnection). PostgreSQL / MySQL not evidenced in your lab files.

13.5 MVC folder structure

Chapter 3. HW 4 Car Rental is the practical MVC piece.

13.6 ASP.NET Core Web APIs

Not evidenced in Labs 01–03 or HW 4. Place on **syllabus verification list**.

13.7 Security

- SQL injection → parameterized queries (Labs 02–03)
- Antiforgery on POST forms (MVC)
- Validate input before DB writes

13.8 Transactions / Unit of Work

Transactions: brief in Chapter 7. **Unit of Work:** not evidenced — verify.

13.9 Testing, deployment

Not in provided lab PDFs — verify.

13.10 lecture6 stubs

Student.cs / StudentDAL.cs match the Lab 02 pattern (model + DAL). Files were empty in the workspace — treat as unfinished practice, not alternate syllabus.

Syllabus verification checklist (ask teacher)

- [ ] Is async/await on the written exam?
- [ ] Are DI lifetimes examinable?
- [ ] ViewModels required?
- [ ] EF Core ever introduced later?
- [ ] Web API?
- [ ] Auth?
- [ ] Only ADO.NET + MVC + Bootstrap?

Chapter 14: Full Integrated Car Rental Reference

Two tracks are shown on purpose:

1. **HW 4 track (confirmed assignment):** in-memory lists, no EF/DB/repos/interfaces.
2. **Architecture track (exam expansion):** ADO.NET repository + optional DI/async — **not** what HW 4 allowed, but connects Chapters 7–10.

Mark configuration you must supply yourself with `YOUR_*`.

14.1 Folder structure (architecture track)

```
CarRental/
  Program.cs
  appsettings.json        # YOUR connection string here
  Controllers/CarsController.cs
  Models/Car.cs
  ViewModels/AvailableCarsViewModel.cs
  Data/ICarRepository.cs
  Data/CarRepository.cs
  Views/
```

```
  _ViewStart.cshtml
  _ViewImports.cshtml
  Shared/_Layout.cshtml
  Shared/_AdminLayout.cshtml
  Cars/Index.cshtml
  Cars/Booked.cshtml
  Cars/Create.cshtml
 wwwroot/
```

14.2 Model

```
// Models/Car.cs
using System.ComponentModel.DataAnnotations;

namespace CarRental.Models
{
  public class Car
  {
    public int Id { get; set; }

    [Required, StringLength(100)]
    public string Name { get; set; } = string.Empty;

    [Range(1, 1000000)]
    public decimal PricePerDay { get; set; }

    public bool IsBooked { get; set; }
  }
}
```

14.3 HW 4 controller (in-memory — persistent only until restart)

```
public class CarsController : Controller
{
  private static List<Car> availableCars = new()
  {
    new Car { Id = 1, Name = "Alto", PricePerDay = 3000m },
    new Car { Id = 2, Name = "Corolla", PricePerDay = 6000m },
    new Car { Id = 3, Name = "Civic", PricePerDay = 8000m }
```

```
    };
    private static List<Car> bookedCars = new();

    [HttpGet]
    public IActionResult Index() => View(availableCars);

    [HttpPost]
    public IActionResult Book(int id)
    {
        var car = availableCars.FirstOrDefault(c => c.Id == id);
        if (car == null) return RedirectToAction(nameof(Index));
        availableCars.Remove(car);
        bookedCars.Add(car);
        return RedirectToAction(nameof(Booked));
    }

    [HttpGet]
    public IActionResult Booked() => View(bookedCars);
}
```

This does not permanently save to a database. Restart clears bookings.

14.4 SQL schema (architecture track — you create DB)

```
-- Run on YOUR SQL Server instance
CREATE TABLE Cars (
    Id INT PRIMARY KEY IDENTITY(1,1),
    Name VARCHAR(100) NOT NULL,
    PricePerDay DECIMAL(10,2) NOT NULL,
    IsBooked BIT NOT NULL DEFAULT 0
);

INSERT INTO Cars (Name, PricePerDay, IsBooked) VALUES
('Alto', 3000, 0), ('Corolla', 6000, 0), ('Civic', 8000, 0);
```

appsettings.json (placeholder):

```
{
  "ConnectionStrings": {
    "Default": "Server=YOUR_SERVER;Database=CarRentalDB;User
Id=YOUR_USER;Password=YOUR_PASSWORD;TrustServerCertificate=True;"
```

```
 }
}
```

14.5 Repository (ADO.NET + async)

```
public interface ICarRepository
{
   Task<List<Car>> GetAvailableAsync();
   Task<List<Car>> GetBookedAsync();
   Task<bool> BookAsync(int id);
   Task AddAsync(Car car);
}

public class CarRepository : ICarRepository
{
   private readonly string _cs;
   public CarRepository(IConfiguration config)
   {
      _cs = config.GetConnectionString("Default")
          ?? throw new InvalidOperationException("Set ConnectionStrings:Default");
   }

   public async Task<List<Car>> GetAvailableAsync()
   {
      return await QueryAsync("SELECT Id, Name, PricePerDay, IsBooked FROM Cars
WHERE IsBooked = 0");
   }

   public async Task<List<Car>> GetBookedAsync()
   {
      return await QueryAsync("SELECT Id, Name, PricePerDay, IsBooked FROM Cars
WHERE IsBooked = 1");
   }

   private async Task<List<Car>> QueryAsync(string sql)
   {
      var list = new List<Car>();
      await using var conn = new SqlConnection(_cs);
      await using var cmd = new SqlCommand(sql, conn);
      await conn.OpenAsync();
      await using var reader = await cmd.ExecuteReaderAsync();
```

```
        while (await reader.ReadAsync())
        {
            list.Add(new Car
            {
                Id = reader.GetInt32(0),
                Name = reader.GetString(1),
                PricePerDay = reader.GetDecimal(2),
                IsBooked = reader.GetBoolean(3)
            });
        }
        return list;
    }

    public async Task<bool> BookAsync(int id)
    {
        const string sql = "UPDATE Cars SET IsBooked = 1 WHERE Id=@id AND IsBooked = 0";
        await using var conn = new SqlConnection(_cs);
        await using var cmd = new SqlCommand(sql, conn);
        cmd.Parameters.AddWithValue("@id", id);
        await conn.OpenAsync();
        return await cmd.ExecuteNonQueryAsync() == 1;
    }

    public async Task AddAsync(Car car)
    {
        const string sql = "INSERT INTO Cars (Name, PricePerDay, IsBooked) VALUES (@n,
@p, 0)";
        await using var conn = new SqlConnection(_cs);
        await using var cmd = new SqlCommand(sql, conn);
        cmd.Parameters.AddWithValue("@n", car.Name);
        cmd.Parameters.AddWithValue("@p", car.PricePerDay);
        await conn.OpenAsync();
        await cmd.ExecuteNonQueryAsync();
    }
}
```

14.6 DI registration + controller

```
// Program.cs
builder.Services.AddControllersWithViews();
builder.Services.AddScoped<ICarRepository, CarRepository>();
```

```
public class CarsController : Controller
{
    private readonly ICarRepository _repo;
    public CarsController(ICarRepository repo) => _repo = repo;

    [HttpGet]
    public async Task<IActionResult> Index()
    {
        var cars = await _repo.GetAvailableAsync();
        var vm = new AvailableCarsViewModel
        {
            Heading = "Available Cars",
            Cars = cars,
            TotalCount = cars.Count
        };
        return View(vm);
    }

    [HttpPost, ValidateAntiForgeryToken]
    public async Task<IActionResult> Book(int id)
    {
        if (!await _repo.BookAsync(id))
            return RedirectToAction(nameof(Index));
        return RedirectToAction(nameof(Booked));
    }

    [HttpGet]
    public async Task<IActionResult> Booked()
        => View(await _repo.GetBookedAsync());

    [HttpGet]
    public IActionResult Create() => View(new Car());

    [HttpPost, ValidateAntiForgeryToken]
    public async Task<IActionResult> Create(Car car)
    {
        if (!ModelState.IsValid) return View(car);
        await _repo.AddAsync(car);
        return RedirectToAction(nameof(Index));
    }
}
```

14.7 Views + layout

Use Chapter 11 _Layout (dark navbar, @RenderBody(), footer, Bootstrap 5.3).
Index: Chapter 5 ViewModel + Chapter 12 cards.
Booked: Chapter 4 table.
Create: Chapter 6 Tag Helper form.

Admin layout example:

```
@{ Layout = "_AdminLayout"; }
```

No layout:

```
@{ Layout = null; }
```

Page script section:

```
@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

14.8 Request flow (architecture track)

```
GET /Cars/Index
  → CarsController.Index (async)
  → ICarRepository.GetAvailableAsync
  → SQL Server SELECT
  → AvailableCarsViewModel
  → Views/Cars/Index.cshtml inside _Layout
  → HTML cards

POST /Cars/Book id=2
  → BookAsync UPDATE
  → 302 Redirect Booked
GET /Cars/Booked
  → table of booked cars
```

14.9 Conceptual vs production

Example	Persistent?
HW 4 static lists	No — RAM only
ADO.NET repository	Yes — SQL Server
Missing real connection string	Will not run until you configure YOUR_*

Do not present in-memory demos as if they permanently save bookings.

Chapter 15: Final Exam Revision

A. One-page cheat sheet

- **MVC:** Model=data, View=HTML, Controller=request handler
- **GET** read / **POST** change; after POST → **Redirect** (PRG)
- @model T declares type; @Model is instance
- **ViewModel** packs multiple values into one @model
- **ADO.NET:** SqlConnection → SqlCommand + parameters → ExecuteNonQuery / ExecuteReader
- **DAL/Repository:** no SQL in controllers
- **SQL injection:** never concatenate user input into SQL
- **Connected:** DataReader | **Disconnected:** DataAdapter + DataTable
- **async Task<IActionResult>** + await repo async methods
- **DI:** AddScoped<ICarRepository, CarRepository>() + constructor injection
- **Layout:** @RenderBody(); sections via @RenderSectionAsync
- **Bootstrap 5:** layout once; cards/tables/navbar/alerts; JS for collapse/modal

B. Essential syntax collection

```
// Action + view + redirect
public IActionResult Index() => View(model);
public IActionResult Go() => RedirectToAction(nameof(Index));
public async Task<IActionResult> Index() => View(await _repo.GetAvailableAsync());

// Razor
@model List<Car>
@foreach (var c in Model) { <p>@c.Name</p> }

// Form + binding
[HttpPost] public IActionResult Book(int id) { ... }
<input type="hidden" name="id" value="@car.Id" />

// Parameterized SQL
cmd.Parameters.AddWithValue("@id", id);

// DI
builder.Services.AddScoped<ICarRepository, CarRepository>();
public CarsController(ICarRepository repo) => _repo = repo;

// Layout sections
@await RenderSectionAsync("Scripts", required: false)
@section Scripts { <script>...</script> }
```

Bootstrap: container, row, col-md-4, card, table table-striped, navbar-dark bg-dark, alert alert-info, badge bg-success, pagination / page-item / page-link.

C. Comparison tables

A	B	Key difference
IActionResult	ViewResult	Interface vs view-only result
ViewBag	ViewData	dynamic vs dictionary; both request-scoped
ViewData	TempData	TempData can survive one redirect
Model	ViewModel	Domain entity vs page DTO

A	B	Key difference
GET	POST	Read vs change state
List<T>	IEnumerable<T>	Concrete mutable list vs enumerable contract
Class	Interface	Implementation vs contract
IoC	DI	Principle vs injection technique
Transient	Scoped	Singleton
Sync	Async	Block wait vs non-blocking I/O wait
Task	Task<T>	No value vs returns T
Layout	Partial	Full shell vs fragment
RenderBody	RenderSectionAsync	Main content vs named slot
Connected	Disconnected	Reader live connection vs DataTable offline
Bootstrap CSS	Bootstrap JS	Look vs interactive components

D. Mock university exam

Part I — MCQs (1 mark each)

1. HW 4 booking should use: (a) GET (b) POST (c) PUT only (d) HEAD

2. Lab 02 data access API: (a) EF Core (b) ADO.NET (c) Dapper (d) Hibernate

3. Prevent SQL injection: (a) concatenate (b) parameters (c) disable DB (d) ViewBag

4. `@model` means: (a) instance (b) type declaration (c) layout (d) route

5. After successful POST Book, best next step: (a) View Index (b) Redirect Booked (c) Content (d) File

6. AddScoped lifetime: (a) one per app (b) one per request (c) every inject (d) never

7. Disconnected Lab 03 uses: (a) only StreamWriter (b) DataTable+Adapter (c) only MVC (d) Redis

8. Dark navbar links should avoid: (a) nav-link (b) conflicting text-dark (c) collapse (d) brand

9. ExecuteNonQuery used for: (a) SELECT lists (b) INSERT/UPDATE/DELETE (c) CSS (d) routing

10. Async controller signature: (a) void (b) Task<IActionResult> (c) Car (d) string

Part II — Short questions

11. Define MVC.

12. Difference List vs IEnumerable.

13. Why DAL/repository?

14. What does @RenderBody() do?

15. Connected vs disconnected in one sentence each.

Part III — Comparisons

16. GET vs POST (table).

17. Model vs ViewModel.

18. ViewBag vs TempData.

Part IV — Code tracing

19. Given HW 4 lists with Alto, Corolla, Civic available; user POSTs Book id=2 twice quickly after first redirect — what is in each list after first successful book?

Part V — Find the error

20. ```csharp public async Task Index() { var cars = _repo.GetAvailableAsync(); return View(cars); }

21. ```csharp
string sql = "SELECT * FROM Products WHERE Name LIKE '%" + keyword + "%'";

Part VI — Coding

22. Write Product class (Lab 02 fields) with constructors.

23. Write AddProduct with parameterized SqlCommand.

24. Write strongly typed Index view loop for List<Car>.

25. Write _ViewStart + navbar brand “Car Rental”.

Part VII — Integration

26. Design flow: Bootstrap form Create Car → validate → ADO.NET insert → redirect Index. List files touched.

27. Explain how to convert HW 4 in-memory Book to SQL UPDATE via repository without EF Core.

Mock exam answer key

1. b 2. b 3. b 4. b 5. b 6. b 7. b 8. b 9. b 10. b

2. Separates Model, View, Controller responsibilities.

3. List is concrete/addable; IEnumerable is for enumeration.

4. Centralize SQL; keep controllers thin; reuse/test.

5. Inserts the view’s main HTML into the layout.

6. Connected: read with open connection (DataReader). Disconnected: fill DataTable, update later via Adapter.

16–18. See section C tables.

19. After first book: available = Alto, Civic; booked = Corolla. Second POST with id=2 finds nothing → redirect Index, lists unchanged.

20. Missing `await` — view receives `Task<List<Car>>` instead of list.

21. SQL injection risk — use parameter + wildcards in value.

22–25. See Chapters 1, 7, 4, 11.

26. `Create.cshtml`, `CarsController.Create` GET/POST, `Car` annotations, `CarRepository.AddAsync`, `Program.cs` DI, `_Layout` Bootstrap form classes, redirect Index.

27. Replace static lists with `ICarRepository.BookAsync` running parameterized UPDATE; register DI; keep same views.

E. Last-minute checklist

I can define: MVC, DAL, parameterized query, ViewModel, layout, model binding, async Task, DI scoped.

I can compare: GET/POST, List/IEnumerable, connected/disconnected, class/interface, IoC/DI, RenderBody/Section.

I can trace: Browser → route → controller → DAL → SQL → view → HTML; PRG booking.

I can write from memory: Car model; CarsController Index/Book/Booked; Razor foreach table/cards; ProductDAL Add/GetAll; `_Layout` navbar; Bootstrap card+table classes.

I know HW 4 rules: in-memory, Bootstrap 5.3, no EF/DB/repos/interfaces for that assignment.

I know lab stack: streams → ADO.NET SQL Server DAL → disconnected/JSON → MVC Car Rental.

Completeness review (against master prompt)

Required chapter	Present
1 C# foundations	Yes
2 HTTP	Yes
3 MVC	Yes
4 Razor	Yes
5 ViewModels	Yes
6 Forms	Yes
7 Database ADO.NET	Yes (EF marked not taught)
8 Repository	Yes
9 Async	Yes
10 DI	Yes
11 Layouts/sections	Yes
12 Bootstrap	Yes
13 Other / verification	Yes
Integrated Car Rental	Yes
Cheat sheet / syntax / comparisons / mock / checklist	Yes

Evidence honesty: ChatGPT chat history not available; Labs 01–03 + HW 4 used as primary course evidence.

End of Handbook

Practice writing code on paper. Re-do Lab 02 CRUD and HW 4 views without looking. Good luck — Professor Shuja's Web Technologies (SEF24).